# Chaitin–Gödel I in Basic Recursive Arithmetic: an object-level diagonal program and an internal implication

## 1 Chaitin's argument, recalled

Chaitin's proof of Gödel's first incompleteness theorem [1, 2] runs as follows. Fix a sound theory $T$ that interprets enough arithmetic, and fix a universal partial computable interpreter $U$ with a Kolmogorov-complexity bound $K_U(x) := \min\{|p| : U(p) \downarrow = x\}$. There is a constant $c$ (depending only on $T$ and $U$) such that, for every $L^\star$ and every integer $x'$,

$$T \vdash K_U(\underline{x'}) > L^\star \quad \Longrightarrow \quad T \text{ is inconsistent, provided } L^\star > c + \log L^\star.$$

The contradiction is exhibited by a single short program $g_{L^\star}$: enumerate $\mathsf{thmT}(0), \mathsf{thmT}(1), \ldots$ until a proof of the form $\mathsf{code}K_U(\underline{?}) > L^\star$ is found, then output the witnessed integer $x'$. The size of $g_{L^\star}$ is $c + \log L^\star$, strictly below $L^\star$, so the very witness $x'$ that $T$ proves to be $K_U$-incompressible is itself $g_{L^\star}$-output — a contradiction to the size lemma internal to $T$.

## 2 The Basic Recursive Arithmetic statement

The formalisation packages Chaitin's argument as a single closed $T$-derivation: it produces, from any putative $T$-proof that $\mathsf{thmT}\, w$ asserts incompressibility of its own read-back, a $T$-proof that a constructed code is a proof of $0 = 1$. Two design choices make the statement sharp.

1. The "construct the short program $g_{L^\star}$" step is a genuine Basic Recursive Arithmetic combinator $\mathsf{G} : \mathsf{Fun}_1$, an honest object-level program, not a meta-level term transformer. The constructed proof code is literally $\mathsf{ap}_1\, \mathsf{G}\, w$.
2. The "if … then inconsistent" step is an *internal* Hilbert implication $\Rightarrow$ inside a single $\mathsf{Deriv}$, not a meta-level function $\mathsf{Deriv}\, X \to \mathsf{Deriv}\, Y$. The deduction content lives inside the object logic.

The headline theorem (`T4/CgFunFun1.agda`, `chaitinG1_Fun1_all`) is:

$$(w : \mathrm{Term}) \;\to\; \mathsf{Deriv}\Big(\underbrace{\mathsf{eqF}\,(\mathsf{ap}_1\,\mathsf{thmT}\,w)\,(\mathsf{ap}_1\,\mathsf{Kcode}_{L^\star}\,(\mathsf{ap}_1\,\mathsf{outKdef}_{L^\star}\,w))}_{\text{firing condition at } w}$$
$$\Rightarrow\; \underbrace{\mathsf{eqF}\,(\mathsf{ap}_1\,\mathsf{thmT}\,(\mathsf{ap}_1\,\mathsf{G}\,w))\,\mathsf{codeFalse}}_{\mathsf{ap}_1\,\mathsf{G}\,w \text{ proves } 0=1}\Big).$$

Here $w$ is an *arbitrary* Term: no closure hypothesis, no $\mathsf{isNat}$ integrality predicate, no $\mathrm{Con}(T)$ premise, and no closedness bookkeeping. In words: *for every* $w$, $T$ proves "if $\mathsf{thmT}\, w$ is the

incompressibility statement $\mathsf{Kcode}_{L^\star}$ about its own read-back $\mathsf{outKdef}_{L^\star}\, w$, then the code $\mathsf{ap}_1\,\mathsf{G}\,w$ is a $T$-proof of $0 = 1$".

The antecedent is the firing condition in num-raw form: the subject slot is $\mathsf{num}\, x'$ for an arbitrary Term $x'$, with no integrality required to align with the read-back. Whether $\mathsf{thmT}\, w$ has the form $\mathsf{Kcode}_{L^\star}\,(\_)$ is read off internally by the Basic Recursive Arithmetic function $\mathsf{outKdef}_{L^\star} = \mathsf{decode} \circ \mathsf{projKdef} \circ \mathsf{thmT}$, so the witnessed integer is forced to be $\mathsf{outKdef}_{L^\star}\, w$.

The program $\mathsf{G}$ is obtained by bracket abstraction: the Chaitin construction is a fixed one-variable object-level combinator context, and $\mathsf{G} : \mathsf{Fun}_1$ is its abstraction, with $\mathsf{ap}_1\,\mathsf{G}\,w$ equal to that construction — as a *proved* $T$-derivation, via combinatory completeness, not a definitional identity (`T4/AbsFun1.agda`, `T4/CgFunFun1.agda`). The implication is the deduction-theorem internalisation of the firing step (`T4/CgFalseImp.agda`). Because $\mathsf{G}$ is an honest combinator, the statement holds at an arbitrary $w$ with no closedness side conditions.

# 3 The subtle part: the internal evaluator and majorisation

The conclusion ultimately rests on an *object-level* run of the universal interpreter evalU on the looping program $g_{L^\star}$ embedded in G. This is where the formalisation is delicate, and it is the part worth documenting carefully. Indeed, the main technical novelty of the formalisation is not the diagonal argument itself — a faithful rendering of Chaitin's — but the construction of *object-level fuel majorants* that let evaluator completeness be expressed and proved entirely inside Basic Recursive Arithmetic. The diagonalisation is classical; the Term-valued fuel of §3.2 is not.

## 3.1 Representation of the universal interpreter

Inside the formalisation, evalU is realised as a small CK-machine: configurations carry a control Term together with a continuation stack, and a one-step transition function stepU advances one rule at a time. Running the interpreter for fuel $n$ is iteration of stepU from a starting configuration. A second layer wraps the $\mu$-search loop of $g_{L^\star}$, so that the interpreter's outer behaviour matches Chaitin's informal description "enumerate $\mathsf{thmT}\,0, \mathsf{thmT}\,1, \ldots$ until a fitting proof is found". The rule-by-rule layout of stepU is not what makes the proof delicate; what matters is the *shape* of the completeness statement the proof needs.

## 3.2 Gandy–Howard majorisation for Term-fuel completeness

A first attempt at a completeness statement would say: for every *numeral* fuel $\overline{n}$ large enough, iterating stepU on $g_{L^\star}$ at $\overline{n}$ reaches the final configuration $\mathsf{cfgRT}\,(\mathsf{num}\, x')\, K_0$. Such a statement is useless inside an internal Deriv-argument: thmT, evalU, and stepU all take Terms, not meta-Nats, so the fuel slot of the resulting thmT-fact must itself be a Term. The obstruction is precise: since Theorem 13 internalises only statements about Basic Recursive Arithmetic terms, a completeness theorem quantified over meta-level naturals cannot be fed into the later derivation. The completeness theorem actually needed has the form

$$(x : \mathrm{Term}) \to \mathsf{Deriv}\big(\mathsf{iter}\,\mathsf{stepU}\,(\mathsf{cfgEV}\,(\mathsf{mcode}\, f)\, x\, K_0)\,(\mathsf{fuelG}\, f\, x) = \mathsf{cfgRT}\,(\mathsf{num}\,(f\, x))\, K_0\big),$$

i.e. Term-input/Term-fuel completeness, where $\mathsf{fuelG}\, f$ is a closed Basic Recursive Arithmetic combinator. Proving such a statement by structural induction on the combinator tree of $f$ stalls at the $R$-recursion node: the inner induction would need numerical control over how much fuel the recursive calls consume, but the actual running time is not available at the Deriv-level.

The resolution adapts the classical Gandy–Howard majorisation method [3, 4]. One builds, by primitive recursion on the structure of $f$, a closed Basic Recursive Arithmetic combinator $\mathsf{fuelG}\, f : \mathsf{Fun}_1$ that provably *majorises* the numerical fuel any actual $\mathsf{stepU}$-run consumes at every Term input. The induction-with-substituted-motive then carries the stable shape "the run completes at *any* fuel $\geq \mathsf{fuelG}\, f\, x$", which descends through the $R$-node because $\mathsf{fuelG}$ is itself defined by a recursive scheme that mirrors $f$. The $\mu$-loop wrapping $g_{L^\star}$ requires the same idea one level higher: a majorising combinator $\mathsf{fuelMu}$ controls the outer search depth and is layered on top of $\mathsf{fuelG}$ by the fuel-composition lemma $\mathsf{iter\_add\_T}$. The closed Term fuel that finally appears inside $\mathsf{ap}_1\, \mathsf{G}\, w$ is

$$n_T \;:=\; \mathsf{fuelMu}\, k_{\max}\, k_{\max} \;+\; \mathsf{fuelG}\, k_{\max}\, O, \qquad k_{\max} = \mathsf{gRec}\, O\, (s\, (\mathsf{closeW}\, w)),$$

and the resulting positive $\mathsf{thmT}$-fact

$$\mathsf{thmT}\, \mathsf{cPos} \overset{\mathsf{Deriv}}{=} \mathsf{coderunProg}\, g_L\, n_T = s\, x'$$

follows from this run via Theorem 13 (singulary). Without majorisation there would simply be no Term to put in the fuel slot.

### 3.3 Closing the chain

With Term-fuel completeness in hand, the operational core closes the chain

$$\mathsf{evalU}\, (\mathsf{parse}\, \mathsf{gLnameDef})\, n_T = s\, (\mathsf{outKdef}_{L^\star}\, k_{\max})$$

through seven internal segments (the module $\mathsf{ChainKdef}$), glued by $\mathsf{iter\_add\_T}$. The positive $\mathsf{thmT}$-fact above is its internalisation via Theorem 13. The negative leg $\mathsf{thmT}\, (\mathsf{cmp}\, \mathsf{outerWrap}\, \mathsf{cSize}) = \mathsf{cNeg}\, \mathsf{coderunProg}\, g_L\, n_T = s\, x'$ is obtained from the firing condition by two $\mathsf{thmT\_at\_sb}$ substitutions and an $\mathsf{encoded\_mp}$ against the size atom. The clash is then a single $\mathsf{encoded\_mp}$ / $\mathsf{encoded\_exfalso}$ pair, internal throughout under the $\Rightarrow$-context.

### 3.4 No consistency hypothesis

Chaitin's original argument concludes "$T$ is inconsistent" (in the metatheory). The formalisation strengthens this: the conclusion is an *internal* implication, a single Basic Recursive Arithmetic derivation that, on the firing condition, the constructed code $\mathsf{ap}_1\, \mathsf{G}\, w$ is a $T$-proof of $0 = 1$. No $\mathrm{Con}(T)$ premise is consumed, no $\mathsf{isNat}$ integrality predicate is consumed, and the firing condition is stated in $\mathsf{num}$-raw form.

This is the precise interface offered to the later surprise-Gödel-II descent: that descent will *not* need to reprove any evaluator completeness, and will use the implication purely as a black-box arrow from a $\mathsf{Deriv}$-witness of the firing condition to a $\mathsf{Deriv}$-witness of $\mathsf{codeFalse}$. The Term-fuel completeness work paid for here — and in particular the Gandy–Howard machinery of §3.2 — is invisible to the descent. The contrast with consistency is sharp: the implication itself consumes no $\mathrm{Con}(T)$, but the surprise-Gödel-II argument will consume internal consistency *after*, and only after, the implication has been discharged.

**Remark 1** (Lineage)**.** *Although the result formalises Chaitin's theorem, its* shape *— an object-level proof predicate, a constructed code, and an internal implication $A \to \mathsf{Proof}(p, 0 = 1)$ — places it closer to the formal-provability tradition of Hilbert–Bernays [5], Löb [6], and Feferman [7], and to its modern mechanisations [8], than to Chaitin's original metatheoretic presentation. What is proved here is not the metatheorem "$T \vdash A$ implies $T$ inconsistent" but the single internal derivation of the corresponding implication, in the style of those derivability-condition analyses.*

## 4 Files

- `T4/AbsFun1.agda` — the bracket-abstraction DSL in one variable, and the combinatory-completeness evaluation (a proved Deriv).
- `T4/CgFunFun1.agda` — the genuine $\mathsf{Fun}_1$ diagonal G and the headline chaitinG1_Fun1_all.
- `T4/CgFalseImp.agda` — the internal implication (deduction-theorem internalisation) and its fresh-witness instance.
- `T4/CloseW.agda` — closeW and the structural lemmas it supports.

All files typecheck under `--safe --without-K --exact-split` with no holes and no postulates.

## References


[1] G. J. Chaitin, "Computational complexity and Gödel's incompleteness theorem," *ACM SIGACT News*, no. 9, pp. 11–12, 1971.

[2] G. J. Chaitin, "Information-theoretic limitations of formal systems," *Journal of the ACM*, vol. 21, no. 3, pp. 403–424, 1974.

[3] W. A. Howard, "Hereditarily majorizable functionals of finite type," in A. S. Troelstra (ed.), *Metamathematical Investigations of Intuitionistic Arithmetic and Analysis*, Lecture Notes in Mathematics, vol. 344, Springer, 1973, pp. 454–461.

[4] R. O. Gandy, "Proofs of strong normalization," in J. P. Seldin and J. R. Hindley (eds.), *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, Academic Press, 1980, pp. 457–477.

[5] D. Hilbert and P. Bernays, *Grundlagen der Mathematik*, vol. II, Springer, 1939.

[6] M. H. Löb, "Solution of a problem of Leon Henkin," *Journal of Symbolic Logic*, vol. 20, no. 2, pp. 115–118, 1955.

[7] S. Feferman, "Arithmetization of metamathematics in a general setting," *Fundamenta Mathematicae*, vol. 49, pp. 35–92, 1960.

[8] L. C. Paulson, "A mechanised proof of Gödel's incompleteness theorems using Nominal Isabelle," *Journal of Automated Reasoning*, vol. 55, no. 1, pp. 1–37, 2015.